

포인터란 데이터가 저장된 메모리의 실제 '주소값'을 저장하는 특수한 변수이다.

예를 들어 `int num = 10;` 라는 코드는 `num`이라는 정수형 변수에 10이라는 정수값을 가진다는 것을 의미한다.

이때 `num`이라는 변수에 `&`를 더한 `&num`은 변수 `num`이 저장된 메모리 주소를 의미한다.

메모리 주소 : 컴퓨터가 RAM과 같은 메모리 내에 데이터나 명령어가 저장된 특정 위치를 식별하기 위해 사용하는 고유한 식별자(숫자)

이 주소는 1000이라는 값을 가질 수도 있고 83271...처럼 예측 불가능한 값을 가질 수 있다.

또한 반대로 주소를 통해 그 주소에 저장된 값을 사용하기 위해서는 `*`라는 연산자가 필요하다.

만약 `num`이 포인터라면 `num`이라는 주소가 가리키는 값은 `*num`으로 표현할 수 있다.

포인터의 가장 기본적인 형태는 아래와 같다.

```
int num = 10;
```

```
int *ptr;
```

```
ptr = &num;
```

여기서 `num`은 정수값 10을 가지는 변수이고 `ptr`은 정수형 포인터로 선언되었다.

다음 줄에 `ptr = #`은 `ptr`이라는 포인터에 `num`의 주소를 넣겠다는 뜻이다.

반대로 `ptr`이 뜻하는 값에 `num`의 값을 입력하고 싶다면 `*p = num;`으로 표현이 가능하다.

```
int num = 10;
```

```
int a = 20;
```

```
int *p = &a; // 포인터 p가 변수 a의 주소를 가리키도록 초기화
```

```
*p = num; // a의 값이 20에서 10으로 변경됨
```

위와 같이 포인터가 올바른 주소를 가리키고 있다면 그냥 변수처럼 덮어쓰기도 가능하다.

1000	1001	1002	1003	1004	1005	1006
1007	1008	1009	1010	1011	1012	1013

이것을 그림과 같이 표현하면 다음과 같다.

정수형 Num 변수가 1000~1003 의 주소를 가진다면 1004~1007 주소에 ptr 라는 포인터가 1000 번지 주소의 값을 가지는 것이다.

아래는 이를 다른 그림으로 표현한 것이다.

Num(정수형 변수)



주소 1000

Ptr(포인터)



Num 은 10 이라는 값을 가지고 1000 번지 주소에 있으면 Ptr 은 1000 번지 주소를 가리킨다.

*Ptr 은 Ptr 이 가리키는 주소의 값인 10 을 나타내는 표현이라는 뜻이다.

즉, 다음과 같은 코드를 실행하면

```
printf("%d\n", Num);
```

```
printf("%d\n", *Ptr);
```

둘 다 10 이라는 값을 출력한다.

이때 가장 포인터가 다른 것과 다른 특별한 이유는 다음과 같다.

```
Num = 10;
```

```
A = Num;
```

```
A = 20;
```

이상태로 A 와 Num 을 출력하면 20 과 10 이 출력이 된다.

하지만

```
int Num = 10;
```

```
int *A;
```

```
A = &Num;
```

```
*A = 20;
```

이 상태로 A 와 Num 을 출력하면 20 과 20 이 출력된다.

이것을 보고 포인터의 값을 초기화면 그 포인터가 가지는 주소값도 변경된다는 것을 알 수 있다.

그렇기에 함수에서 원본 변수를 수정하기 위해서는 포인터를 사용하는 것이 가장 이상적인 방법이다.

예를 들어 다음과 같은 코드를 보자.

```
void change(int *p)
```

```
{
```

```
    *p = 100;
```

```
}
```

```
int main()
```

```
{
```

```
    int num = 10;
```

```
    change(&num);
```

```
    printf("%d", num);
```

```
}
```

이 코드에서 num 은 10 으로 초기화된 후 num 의 주소를 change 함수에 보냈다.

하지만 포인터를 쓰지 않았다면 change 함수에서 아무리 num 을 수정해도 결국 main 에서의 num 은 10 그대로이다.

포인터를 사용해서 매개변수로 받았기에 별다른 반환값이 없이 num 의 값을 change 함수에서 변경할 수 있는 것이다.

저 함수의 출력값은 10 이 아니라 100 이다

포인터를 사용하면 좋은 이유는 함수 부분에서 말고도 더 있는데 그중에서 대표적인 것은 바로 배열을 효율적으로 정리할 수 있다는 점이다.

특히 배열은 포인터와 매우 밀접한 관계가 있는데

int arr[3] = {10, 20, 30}; 에서 배열 이름 arr 은 대부분의 상황에서 배열의 첫 번째 요소의 주소처럼 사용된다. (arr == &arr[0])

그렇기에 저 배열에서 printf("%d", *arr); 은 arr 배열의 첫번째 값(0)인 10 을 출력하는 것이다.

또, 두번째 값(1)을 출력하기 위해서는 arr[1]을 사용하지 않고 *(arr + 1)을 사용하면 20 을 나타낸다.

이 관계를 이해하면 배열은 그 자체로 주소를 나타내므로 위에서 말한 함수에 배열을 보낼때는 따로 &를 붙여서 주소임을 알려줄 필요가 없다.

하지만 여기서 알아야 할 점은 포인터에서 p + 1 은 메모리 주소를 1 바이트 이동하는 것이 아니라, 변수형의 크기만큼 이동한다는 점이다.

예를 들어 int 형이라면 int 의 크기(4 바이트)만큼 이동한다.

그래서 p 가 1000 번지 주소를 가리키고 그 주소가 가지는 값이 int 형이라면 p+1 은 1001 이 아니라 1004 이다.

배열이 주소를 나타낸다는 것이 배열과 포인터가 완전히 같다는 것은 아니다.

```
int arr[5];
```

```
int *p;
```

예를 들어 위 코드에서 arr 은 실제 배열이고, p 는 주소를 저장하는 포인터 변수이다.

그래서 p = arr;은 가능하지만 arr = p;는 불가능하다. (이 차이 중요)

이처럼 포인터에 대한 기본 개념을 알았으니 이를 확장한 몇가지 내용을 공부할 것이다.

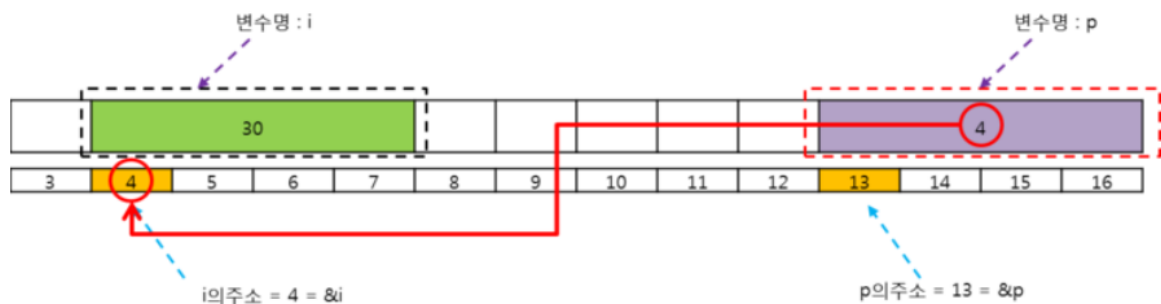
그 첫번째 내용은 이중포인터(double pointer)이다.

이중 포인터는 결국 포인터 변수이다.

매우 쉬운 내용이다.

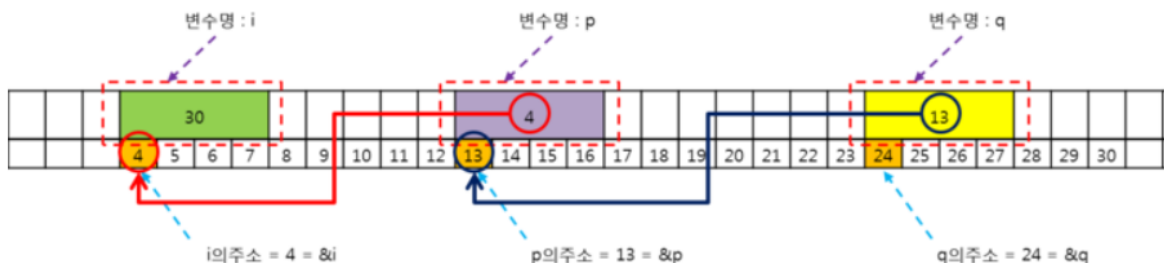
포인터가 일반 변수의 주소값을 가진다면, 이중포인터는 포인터의 주소값을 가진다.

```
int i = 30;  
int* p = &i;
```



예시 사진을 보면 i 는 정수형 변수로 30 이라는 값을 가지고 포인터 변수인 p 는 i 의 주소를 가지므로 *p 도 30 이다.

```
int i = 30;  
int* p = &i;  
int** q = &p;
```



이때 여기서 이중포인터인 q 가 등장하면 q 는 p 의 주소를 가진다.

그렇기에 **q 의 값은 q 가 가리키는 주소인 포인터 *p 를, 거기서 *p 가 포인터이므로 p 가 가리키는 주소인 i 의 값을 나타낸다.

즉, 포인터 **q 가 가리키는 값은 30 인 것이다.

이게 끝이다.

그냥 포인터를 한번 더 사용한 것 뿐인게 이중 포인터인 것이다.

그렇다면 이 이중 포인터를 굳이 사용하는 이유가 무엇인지 궁금해졌다.

이중 포인터가 사용되는 이유는 앞선 시간에 사용했던 동적 메모리 할당으로 2 차원 배열을 만들 때 이중 포인터를 사용해야하고, 함수에서 포인터를 입력받으면 그것이 나타내는 값을 변경할 수 있다고 했는데, 이처럼 포인터를 직접 변경하기 위해 함수로 보낼 때는 이중포인터를 보내면 된다.

이런 방식 말고도 여러 곳에서 이중 포인터가 사용되기에 절대 놓치고 가면 안되는 개념인 것이다.

다음으로 공부할 내용은 함수 포인터(function pointer)이다.

그냥 포인터가 일반 변수의 주소를 저장하는 것이라면 함수 포인터는 함수의 주소를 저장하는 변수이다.

일반 변수의 주소를 저장할 수 있는 것처럼 함수의 주소도 저장이 가능한 것이다.

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

예를 들어 위 코드에서 add 라는 함수가 있다.

여기서 `int (*fp)(int, int);` 라는 함수 포인터를 선언한 후 `fp = &add;` 를 하거나 배열처럼 `fp = add;` 를 해서 fp 라는 포인터 변수에 add 라는 함수의 주소를 저장할 수 있다.

add 함수를 fp 를 이용해서 실행하는 방법은 다른 것과 마찬가지로

`int result = fp(10, 20);` 또는 `int result = (*fp)(10, 20);` 이 있다.

실행 값은 30 이다.

여기서 일반 포인터와 다른 점이라면 *fp 가 아니라 (*fp)처럼 괄호가 있다는 것이다.

괄호가 없다면 그것은 함수 포인터가 아니라 int* 를 반환하는 함수가 되어버리면서 꼬여버린다.

또한 일반적인 포인터와 달리, 함수 포인터는 데이터가 아니라 코드의 위치를 가리킨다.

위에 있는 포인터의 그림처럼 포인터가 배열의 시작 부분을 가리키듯이 함수 포인터도 함수의 코드의 시작 부분을 가리키는 것이다.

또한 함수 포인터를 통해 메모리를 할당하거나 회수하는 것이 불가능하다는 점이 있다.

함수 포인터의 활용 방법은 나중에 배우게 될 콜백 함수, 정렬 함수, 함수 테이블 등이 있으며, 함수를 또 다른 함수의 인자로 전달할 수 있다는 점이 가장 중요한 점이다.

마지막으로 공부할 내용은 구조체 포인터(struct pointer)라는 것이다.

구조체는 얼마전 시간에 배운 내용으로 struct 를 이용하는 것인데, 이 구조체도 포인터로 가리킬 수 있다는 점이 정말 코딩의 세계는 생각하기 나름인 것 같다.

```
struct Student
{
    int age;
    int grade;
    char name[10];
};
```

예를 들어 위와 같은 구조체가 있을 때, 아래와 같이 포인터를 사용할 수 있다.

```
int main()
{
    struct Student * person
}
```

그냥 일반 자료형 포인터 선언과 똑같다.

여기서 선언된 구조체 포인터 person 을 통해 멤버에 접근할 수 있다.

구조체의 멤버 변수를 사용하는 방법은 다음과 같다.

```
int main()
{
    struct Student people;
    struct Student *person;
    person = &people;
    (*person).grade = 10;
```

```
printf("%d", (*person).grade);
}
```

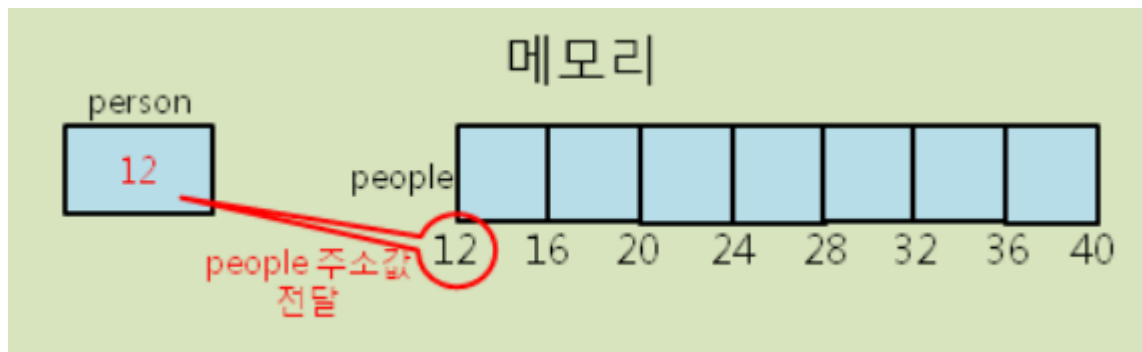
먼저 Student 구조체를 정의하고 구조체 자료형 `people` 과 `person` 포인터 변수를 선언한다.

그 후 `people` 의 주소값을 `person` 에 저장한 후 `grade` 에 10 을 입력하고 이를 출력하면 10 이 출력된다.

하지만 여기서 현재 출력하는 형태는 참조연산자라 하고 평소에 내가 사용하던 구조체에 포인터를 합친 방법인데, 간접참조연산자라는 더 간단한 형태가 존재한다.

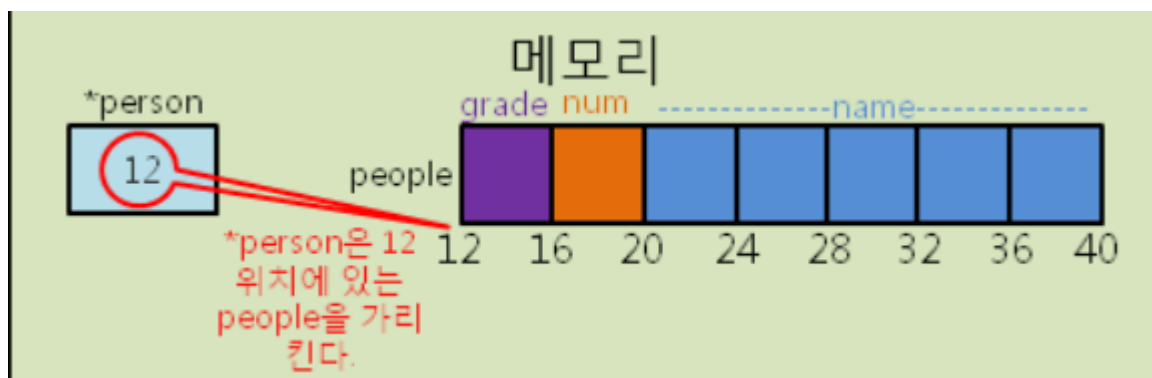
`(*person).grade` 대신 `person->grade` 를 써도 같은 값을 출력한다.

위 코드를 더 이해하기 쉽게 이미지를 활용해서 설명하면 다음과 같다.



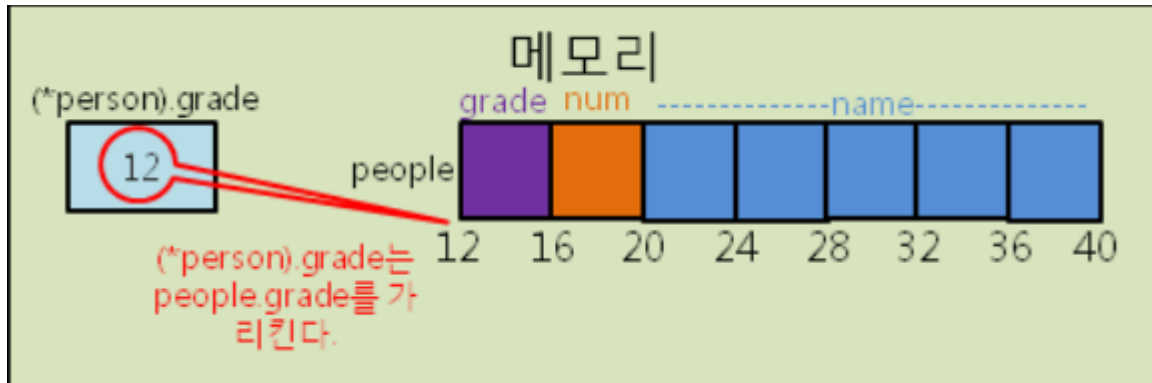
위 그림이 메모리 상에서 `person` 포인터 변수에 `people` 주소값을 저장하는 부분의 모습이다. (`person = &people;`)

멤버 변수를 사용하는 부분의 모습은 아래 사진이다.



위 그림은 일반 포인터의 개념과 똑같이 `person` 이 참조연산자를 이용하여 `people` 의 위치(12)를 가리키는 모습이다.

이번에는 참조를 통해 멤버 변수를 가리킨 모습이다.



이를 통해 (*person).grade 는 people.grade 와 동일하다는 것과 이미지 속에서 보라색 부분의 공간을 사용한다는 것을 알 수 있다.

그렇다면 간접참조연산자(->)는 무엇일까?

이는 참조연산자와 형제되는 연산자라 한다.

차이점이라면 참조 연산자가 자기 자신 구조체의 직접적인 멤버 변수를 사용할 때 사용되는 기호라면 간접참조연산자는 이름에서 알 수 있듯이 자기 자신멤버가 아닌 참조된 구조체의 멤버를 가리킨다는 점이 있다.

즉, 구조체 포인터에서 사용되며 이를 통해 자칫 복잡해질 수 있는 코드를 보다 간단하고 보기 편하게 간략하게 만들어준다.

다시 말해, 구조체가 크다면 함수에 구조체 전체를 복사하는 것보다 주소만 전달하는 것이 효율적이라는 것이다.

이를 이해했다면 grade 뿐만 아니라 age 와 name 도 간접참조연산자를 통해 표현할 수 있다는 것을 알 수 있다.

나는 이번 공부를 통해 포인터는 단순히 메모리 주소를 저장하는 특별한 변수라는 개념에서 끝나는 것이 아니라, C 언어의 여러 기능과 밀접하게 연결되어 있다는 것을 알게 되었다.

처음에는 변수의 주소를 저장하고 그 주소를 통해 값을 가져오는 단순한 기능이라고 생각하고, 시험에는 이렇게만 나온다 하니 그렇게만 배웠었다.

하지만 비록 과제 때문이라지만 스스로 공부를 진행하면서 배열, 함수, 동적 메모리, 이중 포인터, 함수 포인터, 구조체 포인터 등 다양한 곳에서 포인터가 활용된다는 것을 알게 되었고, 사용 방법까지 이해할 수 있었다.

특히 포인터를 이용하면 함수에서 원본 변수의 값을 직접 변경할 수 있고, 배열을 주소와 연결하여 효율적으로 사용할 수 있다는 점은 알고 있었다

하지만 이중 포인터를 통해 포인터 자체를 변경할 수 있고, 함수 포인터를 통해 함수의 주소를 저장하여 다른 함수에 전달할 수 있다는 점에서 포인터가 단순한 변수의 주소를 저장하는 기능을 넘어 C 언어의 다양한 기능을 연결해 주는 중요한 개념이라는 것을 처음 알게 되었다.

이런 점들이 정말 인상깊었다.

나는 코딩의 세상에 새발의 피 정도였다는 것을 깨달으면서도, 아직 새롭게 배움의 시간을 가질 수 있는 기회가 무궁무진하다는 사실이 나를 더 분발시켰다.

구조체 포인터를 공부하면서는 포인터가 단순한 기본 자료형뿐만 아니라 구조체와 같은 사용자 정의 자료형에도 적용될 수 있다는 것을 이해하게 되었다.

특히 예전에 생성형 ai 에게 코드를 만들어 달라고 했을 때는 ->가 뭐인지도 몰랐고 그냥 이렇게 있구나 싶었지만, (*person).grade 와 person->grade 가 같은 의미를 가진다는 것을 통해 포인터를 사용하면서 코드의 표현을 더욱 간단하게 만들 수 있다는 점도 알게 되었다.

이번 공부를 통해 포인터의 기본적인 동작 원리와 활용 방법을 이해할 수 있었지만, 아직 콜백 함수 등 포인터가 실제로 활용되는 부분은 더 깊이 공부할 필요가 있다고 생각한다.

앞으로 포인터에 대한 더 깊은 수업이 있다고 들었기에, 수업 내용을 그저 암기하는 것이 아니라 메모리의 구조와 주소의 이동 과정을 직접 생각하면서 코드를 작성한다면, C 언어를 더욱 깊이 이해할 수 있을 것 같다.

이번 공부를 통해 포인터에 대한 기본적인 이해를 갖추었으며, 앞으로 동적 메모리와 자료구조를 공부할 때에도 포인터의 개념을 바탕으로 더욱 깊이 이해해 보고자 한다.